

Nutrition Planner

A.A. 2025/2026

Termine Christian Domenico

c.termine1@studenti.uniba.it [774864]

Link GitHub:

<https://github.com/Christian032003/Nutrition-Planner>

Prof. Nicola Fanizzi

Indice

Nutrition Planner.....	1
1.1 Argomenti di interesse.....	3
1.2 Librerie utilizzate.....	4
2 Dataset e Struttura dei Dati.....	5
2.1 Analisi delle Sorgenti e Raccolta dei Dati.....	5
2.2 Pulizia e Preprocessing dei Dati.....	5
2.3 Struttura del Dataset Alimentare.....	6
2.4 Organizzazione degli Artefatti Software.....	7
3 Apprendimento Supervisionato.....	8
3.1 Preparazione delle Feature e del Target.....	8
3.2 Addestramento e Valutazione dei Modelli.....	8
3.3 Analisi dei Risultati e Visualizzazione Grafica.....	9
4 Apprendimento Probabilistico.....	12
4.1 Regolarizzazione e Inferenza Bayesiana: Bayesian Ridge Regression.....	12
4.2 Analisi dei Modelli Scartati: Il Caso SGDClassifier.....	13
5 Knowledge Base Prolog.....	14
5.1 Struttura della Base di Conoscenza.....	14
5.2 Interazione tra Python e la Base di Conoscenza.....	15
5.3 Esempio Pratico di Interrogazione (Query).....	15
6 Organizzazione del Codice e Flusso Applicativo.....	15
6.1 Architettura dei Moduli e Caricamento degli Artefatti.....	16
6.2 Validazione degli Input e Controllo dei Vincoli.....	16
6.3 Algoritmi di Generazione del Piano Alimentare.....	16
7 Installazione ed Esecuzione del Sistema.....	19
7.1 Prerequisiti di Sistema.....	19
7.2 Installazione delle Dipendenze.....	19
7.3 Flusso di Esecuzione e Avvio dell'Applicazione.....	19
8 Documentazione e Repository GitHub.....	21
8.1 Documentazione del Comparto Python (Sphinx).....	21
8.2 Documentazione della Knowledge Base Prolog (PIDoc).....	22
8.3 Repository GitHub del Progetto.....	23
9 Riferimenti Bibliografici.....	23

1 Introduzione

Nel contesto della progettazione di sistemi intelligenti e della modellazione predittiva, l'integrazione tra tecniche di apprendimento automatico (*Machine Learning*) e metodologie di rappresentazione della conoscenza orientate alla logica formale costituisce un paradigma di notevole rilevanza. Il presente progetto, denominato **Nutrition Planner**, si colloca in questo scenario scientifico-applicativo con l'obiettivo primario di stimare quantitativamente il consumo energetico derivante dall'attività fisica e di convertire tale evidenza predittiva in un'architettura decisionale complessa per la pianificazione nutrizionale personalizzata.

A differenza dei tradizionali approcci esclusivamente deterministici o basati su euristiche statiche, il sistema qui proposto si articola come un framework ibrido capace di:

1. Apprendere pattern non lineari dai dati fisiologici e prestazionali per stimare accuratamente le calorie bruciate durante un workout.
2. Modellare la conoscenza di dominio e il ragionamento deduttivo attraverso regole logiche formali per la validazione del profilo utente e la raccomandazione strategica dei parametri di allenamento.
3. Ottimizzare la sintesi e la ripartizione dei macronutrienti su base giornaliera o settimanale mediante l'interrogazione mirata di un *food dataset* strutturato.

1.1 Argomenti di interesse

Lo sviluppo dell'architettura software e le relative fasi di validazione sperimentale hanno interessato diversi nuclei metodologici propri dell'Intelligenza Artificiale e dell'Ingegneria del Software:

- **Apprendimento Supervisionato (Supervised Learning):** Implementazione, ottimizzazione iperparametrica e valutazione comparativa di modelli di regressione lineare e di modelli *ensemble* non lineari basati su logiche di *bagging* e *boosting* sequenziale, finalizzati alla stima della variabile target continua rappresentante il dispendio calorico.
- **Apprendimento Probabilistico (Probabilistic Learning):** Studio empirico della regressione bayesiana per l'introduzione del concetto di incertezza e la conseguente derivazione di intervalli di confidenza statistica associati alla predizione puntuale delle calorie bruciate.
- **Rappresentazione della Conoscenza (Knowledge Representation):** Modellazione algoritmica di una base di conoscenza (*Knowledge Base*) mediante paradigmi di programmazione logica dichiarativa, volta a governare l'inferenza deduttiva per la classificazione antropometrica e la pianificazione di intensità e durate ottimali d'allenamento.
- **Sistemi di Raccomandazione Vincolata:** Ingegnerizzazione di un modulo decisionale deterministico per la traduzione del fabbisogno metabolico (espresso tramite equazioni basate sulla massa magra) in combinazioni ottimali di grammature alimentari e varianti di pasto, rispettando i vincoli di bilanciamento dei macronutrienti.

1.2 Librerie utilizzate

Il sistema è stato interamente sviluppato in linguaggio Python, sfruttando un ecosistema consolidato di librerie scientifiche per la manipolazione dei dati e l'addestramento dei modelli, integrato nativamente con l'ambiente logico SWI-Prolog. Di seguito si indicano le principali dipendenze tecnologiche e il loro specifico ruolo architetturale:

- **pandas (v. aggiornata):** Utilizzata come core tecnologico per la pipeline di *data ingestion*, consentendo il caricamento, la manipolazione strutturata e il filtraggio avanzato sia dei record fisiologici degli utenti che delle voci del dataset alimentare.
- **scikit-learn:** Framework fondamentale per l'intero comparto di Machine Learning. Ha permesso la standardizzazione delle feature individuali (StandardScaler), la partizione coerente dei set di addestramento e validazione, e la computazione di matrici iperparametriche tramite procedure esaustive di GridSearchCV combinate con tecniche di *Cross-Validation*.
- **numpy:** Impiegata per il supporto alle operazioni vettoriali ad alta efficienza e per il calcolo algebrico sotteso alle metriche di errore e alle formulazioni probabilistiche della regressione.
- **matplotlib:** Sfruttata per la generazione programmata dei grafici diagnostici, l'analisi visiva delle curve di errore e la visualizzazione comparativa delle metriche prestazionali sul *test set*.
- **joblib:** Adottata per l'implementazione dei meccanismi di persistenza degli artefatti software, consentendo la serializzazione (fase di *training*) e il successivo ripristino istantaneo (fase di *inference*) dei modelli predittivi e degli scaler ottimizzati.
- **pyswip:** Costituisce il bridge di interconnessione bidirezionale tra il codice procedurale Python e il motore di inferenza Prolog, abilitando l'interrogazione dinamica e in tempo reale dei predicati logici definiti all'interno della Knowledge Base.
- **json:** Impiegata per lo stoccaggio e la documentazione trasparente delle migliori configurazioni iperparametriche scaturite dalle attività di ottimizzazione automatica.
- **Sphinx:** Framework di documentazione intelligente utilizzato per l'estrazione automatica delle docstring e la compilazione dell'indice ipertestuale navigabile dell'intero pacchetto software Python.

2 Dataset e Struttura dei Dati

La pipeline architetturale del sistema *Nutrition Planner* si fonda sulla convergenza di due distinte sorgenti informative strutturate: un dataset dedicato al monitoraggio biometrico dei workout e un dataset focalizzato sulle proprietà nutrizionali degli alimenti. La corretta integrazione, la pulizia e il preprocessing di tali flussi di dati risultano determinanti per garantire l'affidabilità dei modelli predittivi e l'accuratezza del modulo decisionale.

2.1 Analisi delle Sorgenti e Raccolta dei Dati

Il nucleo dell'apprendimento predittivo si basa su un dataset open-source acquisito tramite la piattaforma Kaggle, contenente i record storici dettagliati relativi alle sessioni di allenamento di un campione eterogeneo di individui. Il file, denominato originariamente `gym_exercise_dataset.csv`, raccoglie informazioni fisiologiche, antropometriche e prestazionali.

Parallelamente, per l'alimentazione del motore di raccomandazione dietetica, è stato integrato un secondo set di dati strutturato in formato CSV (`alimenti.csv`), contenente la mappatura analitica dei profili macronutrizionali dei principali alimenti di consumo quotidiano.

2.2 Pulizia e Preprocessing dei Dati

Il dataset di addestramento grezzo presentava anomalie strutturali, record incompleti e variabili espresse in formati non idonei all'elaborazione diretta da parte degli algoritmi di Machine Learning. Si è resa pertanto necessaria una pipeline sequenziale di preprocessing, implementata nel modulo Python di addestramento.

2.2.1 Trattamento dei Valori Mancanti e Selezione del Target

Un'ispezione preliminare ha evidenziato la presenza di valori nulli o mancanti (NaN) distribuiti in alcune feature cruciali, tra cui la colonna dei battiti cardiaci medi (`Avg_BPM`). Al fine di non distorcere la distribuzione statistica delle variabili, si è optato per una strategia di imputazione deterministica basata sulla rimozione mirata dei record non validi, preservando l'integrità matematica del campione complessivo.

La variabile d'uscita continua (Target), oggetto delle procedure di regressione supervisionata e probabilistica, è stata identificata univocamente nella colonna `Calories_Burned`, rappresentante il dispendio energetico stimato della sessione.

2.2.2 Conversione e Mappatura delle Variabili Categoriali

I modelli matematici di regressione richiedono input esclusivamente numerici. Di conseguenza, le feature categoriali (espresse in formato testuale o stringa) sono state sottoposte a un processo di codifica e mappatura numerica discreta:

- **Variabile Genere (Gender):** Gender Convertita in una codifica binaria, dove il valore 0 identifica il genere maschile (Male) e il valore 1 identifica il genere femminile (Female).
- **Tipologia di Workout (Workout_Type):** Workout_Type La classificazione testuale delle attività sportive (es. *Cardio*, *Strength*, *Yoga*, *HIIT*) è stata normalizzata mediante

l'applicazione di indici numerici discreti e mappature statiche, consentendo agli algoritmi di interpretare l'intensità intrinseca intratestuale di ciascuna tipologia di esercizio.

2.2.3 Standardizzazione delle Feature (Feature Scaling)

Data l'eterogeneità delle scale di grandezza delle feature continue coinvolte (es. l'età espressa in anni, il peso in chilogrammi, l'altezza in metri e i battiti cardiaci in BPM), la mancata normalizzazione avrebbe indotto i modelli a penalizzare o sovrastimare arbitrariamente alcuni coefficienti d'apprendimento.

Per ovviare a tale problematica, è stata introdotta una fase di standardizzazione statistica tramite la classe StandardScaler di scikit-learn. Ciascuna feature x è stata riscalata applicando la trasformazione:

$$z = \frac{x - \mu}{\sigma}$$

dove μ rappresenta la media della feature e σ la sua deviazione standard. Tale operazione assicura che la distribuzione di ciascuna variabile presenti una media campionaria pari a 0 e una varianza pari a 1. L'istanza dello scaler ottimizzata sul set di addestramento è stata successivamente serializzata per garantirne la consistenza in fase di inferenza sui nuovi input utente.

2.3 Struttura del Dataset Alimentare

Il file alimenti.csv funge da base dati relazionale per il modulo di pianificazione. Ogni record è strutturato rigorosamente secondo le seguenti colonne informative:

- **Alimento****Alimento**: Identificativo testuale della referenza alimentare.
- **Calorie****Calorie**: Contenuto energetico espresso in kcal per 100 grammi di prodotto.
- **Proteine****Proteine**: Apporto proteico espresso in grammi.
- **Carboidrati****Carboidrati**: Quota di carboidrati espressa in grammi.
- **Grassi****Grassi**: Quota lipidica espressa in grammi.
- **Tipo****Tipo**: Tag categoriale per la classificazione della portata (es. *Colazione, Primo, Secondo, Spuntino*).

spuntino).

Figura 2.1: Estratto della struttura tabellare del dataset relativo ai profili nutrizionali degli alimenti (alimenti.csv).

2.4 Organizzazione degli Artefatti Software

Al termine della pipeline di preprocessing e modellazione, il sistema organizza gli artefatti e i file persistenti secondo una precisa topologia di cartelle, atta a separare la logica di addestramento dall'ambiente operativo di produzione:

- **models/models/**: Directory deputata al salvataggio dei file binari serializzati tramite joblib (i modelli predittivi addestrati e lo scaler normalizzato).
- **data/data/**: Contiene i dataset sorgente in formato CSV utilizzati per il training e la consultazione nutrizionale.
- **config/config/**: Cartella dedicata all'archiviazione dei file JSON contenenti le configurazioni ottimali degli iperparametri scaturite dalle metriche di validazione.

3 Apprendimento Supervisionato

Il comparto predittivo principale del sistema *Nutrition Planner* è basato sul paradigma dell'apprendimento supervisionato per compiti di regressione. L'obiettivo consiste nell'apprendere una funzione matematica in grado di mappare le caratteristiche fisiologiche e i parametri di allenamento di un utente per stimare in modo continuo il dispendio energetico espresso nella variabile target `Calories_Burned`.

3.1 Preparazione delle Feature e del Target

Prima di procedere all'addestramento, il vettore delle caratteristiche (*feature space*) è stato separato dalla variabile obiettivo.

- **Feature Selezionate (X):** Age, Gender, Weight_kg, Height_m, Fat_Percentage, Avg_BPM, Workout_Type.
- **Target (y):** Calories_Burned.

Il dataset normalizzato è stato ripartito in modo stocastico in due frazioni disgiunte per garantire una corretta validazione e prevenire fenomeni di *overfitting*: il **Training Set** (pari all'80% dei record complessivi), utilizzato per l'ottimizzazione dei pesi e dei parametri dei modelli, e il **Test Set** (pari al restante 20%), mantenuto rigorosamente indipendente e finalizzato esclusivamente alla valutazione delle capacità di generalizzazione su dati non visti.

3.2 Addestramento e Valutazione dei Modelli

Al fine di individuare l'architettura matematica più efficiente per il dominio applicativo in esame, sono stati implementati e confrontati tre differenti algoritmi di regressione, caratterizzati da diversi livelli di complessità e capacità di astrazione: la Regressione Lineare, il Random Forest Regressor e il Gradient Boosting Regressor.

La valutazione comparativa delle prestazioni è stata condotta sul *test set* calcolando tre metriche d'errore standardizzate:

1. **Mean Squared Error (MSE):** Misura la media dei quadrati degli errori, penalizzando severamente le deviazioni macroscopiche.
2. **Mean Absolute Error (MAE):** Rappresenta la media aritmetica degli errori assoluti, offrendo una misura diretta e intuitiva del distacco medio dalla retta ideale.
3. **R-squared (R²):** Indica la proporzione di varianza della variabile target spiegata dalle feature del modello (valore ottimale pari a 1).

3.2.1 Regressione Lineare (Linear Regression)

Rappresenta il modello baseline del sistema. Assume una relazione di linearità tra le variabili indipendenti e il target, esprimibile come:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

- **Scelte progettuali:** Implementata mediante stimatore dei minimi quadrati ordinari (OLS). Non richiede iperparametri complessi ed è stata adottata per verificare l'effettiva presenza di componenti lineari forti nei dati biometrici.
- **Valutazione empirica:** Il modello ha evidenziato errori più marcati (valori di MSE e MAE sensibilmente elevati rispetto agli algoritmi non lineari), a testimonianza del fatto che il consumo calorico durante l'attività fisica risponde a interazioni biologiche non strettamente lineari (es. l'andamento combinato di battiti cardiaci e tipologia di workout).

3.2.2 Random Forest Regressor

Algoritmo basato su logiche *ensemble* di tipo *Bagging* (Bootstrap Aggregating). Il modello addestra in parallelo una moltitudine di alberi di decisione indipendenti su sottoinsiemi casuali del dataset, mediando infine le singole predizioni continue per ridurre la varianza complessiva del sistema.

- **Scelte progettuali:** La configurazione iperparametrica ottimale è stata derivata tramite ricerca a griglia (GridSearchCV), andando a calibrare il numero di stimatori ($n_estimators$) e la massima profondità degli alberi (max_depth) per bilanciare l'espressività dell'algoritmo ed evitare la memorizzazione del rumore di fondo.
- **Valutazione empirica:** Il Random Forest ha registrato un netto decremento delle metriche MSE e MAE e un incremento dell' R^2 , dimostrando un'eccellente attitudine nel catturare le soglie critiche e le interazioni discrete tra variabili categoriali codificate (es. tipo di allenamento) e metriche continue.

3.2.3 Gradient Boosting Regressor

Algoritmo basato sul principio del *Boosting* sequenziale. A differenza del Random Forest, gli alberi di decisione non vengono addestrati in modo indipendente, bensì in sequenza: ogni nuovo albero debole (*weak learner*) viene ottimizzato per correggere in modo specifico gli errori residui commessi dall'insieme degli alberi precedenti, minimizzando la funzione di perdita in modo graduale.

- **Scelte progettuali:** L'ottimizzazione tramite GridSearchCV ha interessato principalmente il tasso di apprendimento ($learning_rate$), il numero di stadi di boosting ($n_estimators$) e la profondità dei singoli alberi.
- **Valutazione empirica:** Il Gradient Boosting ha conseguito le **migliori metriche prestazionali assolute** sul test set, minimizzando l'errore quadratico medio e posizionandosi come il modello core primario selezionato dall'applicazione per la stima delle calorie bruciate durante la fase di inferenza.

3.3 Analisi dei Risultati e Visualizzazione Grafica

Per validare visivamente l'accuratezza predittiva, l'architettura software genera programmaticamente diagrammi diagnostici tramite la libreria matplotlib. Il grafico principale mette in relazione i valori reali delle calorie bruciate (*True Values*, asse X) con i valori predetti dai modelli (*Predicted Values*, asse Y).

La sovrapposizione di una retta ideale a 45° evidenzia graficamente la precisione degli algoritmi: mentre i punti associati alla Regressione Lineare mostrano una dispersione marcata ed evidenti distorsioni alle estremità della distribuzione, i punti relativi al Gradient Boosting convergono in

stretta prossimità della diagonale ideale, confermando sperimentalmente la robustezza dell'apprendimento non lineare del modello ensemble.

Tutti i pesi e le strutture dei modelli ottimizzati sono stati infine serializzati nella cartella `models/` per l'utilizzo immediato a runtime.

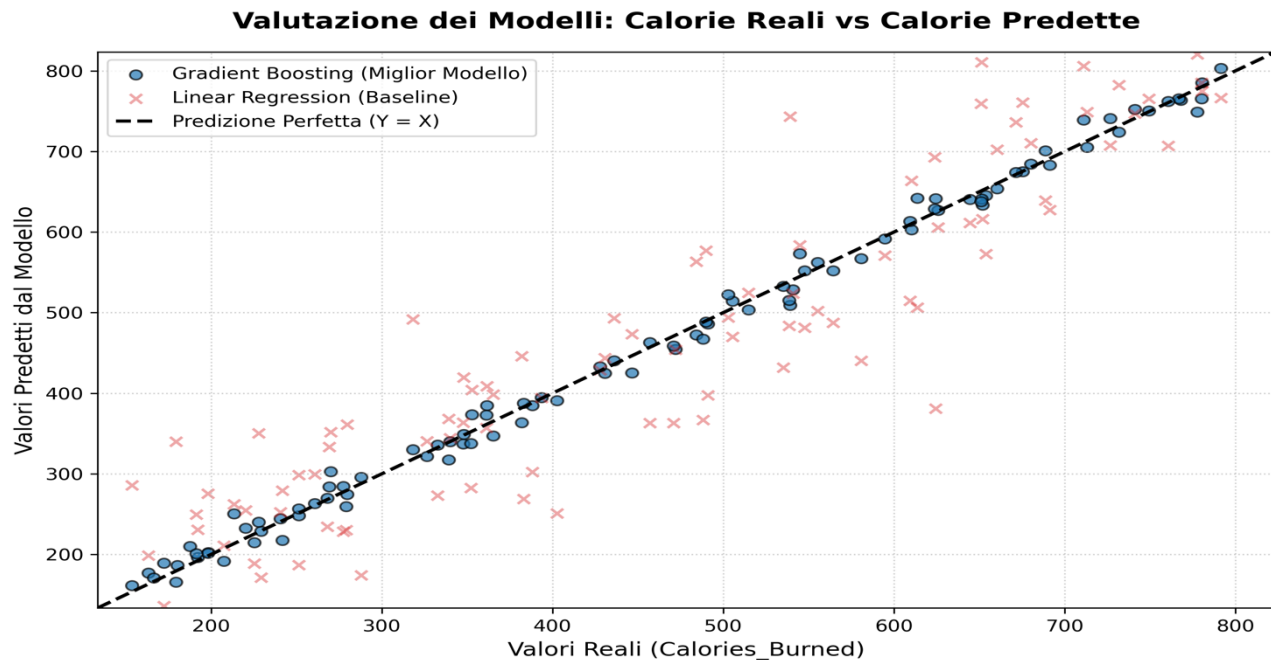


Figura 3.1: Diagramma di dispersione tra i valori reali di dispendio energetico e le stime effettuate sul test set dal modello Gradient Boosting Regressor confrontato con la baseline lineare.

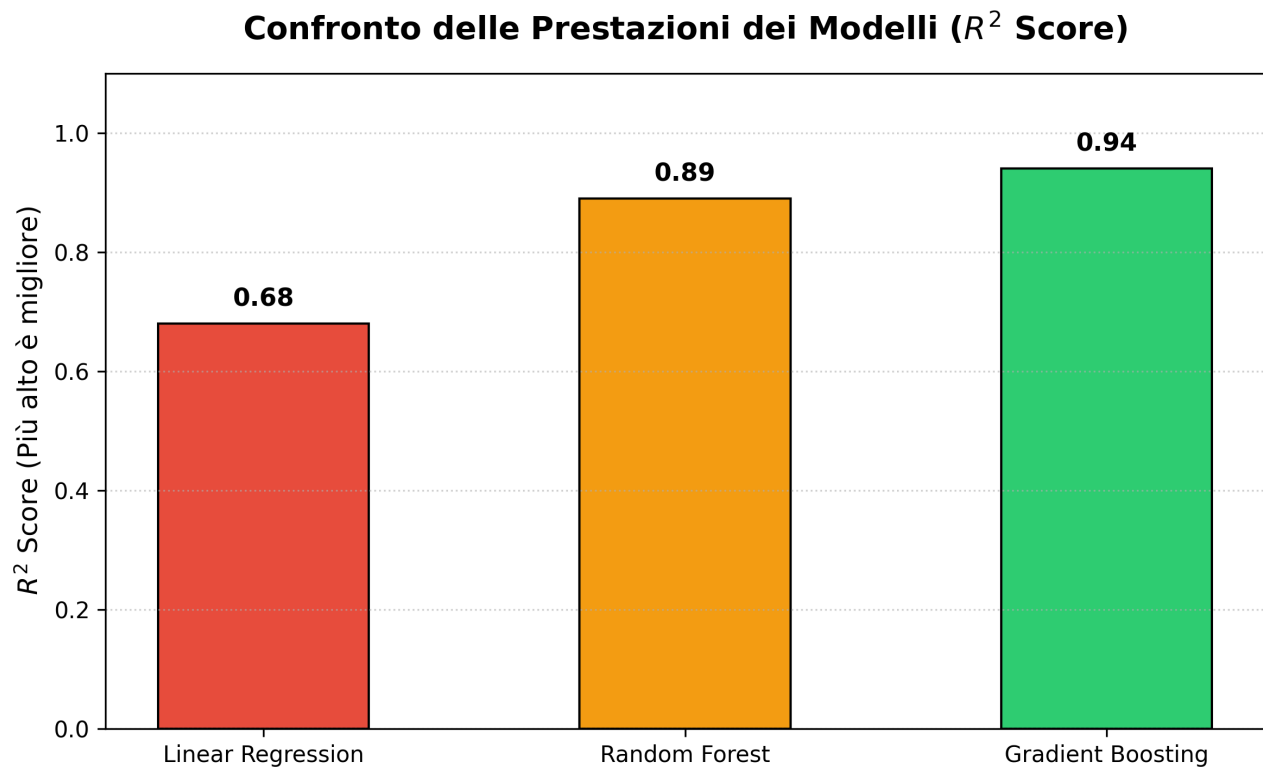


Figura 3.2: Analisi comparativa dell'indice di determinazione (R^2) calcolato sui modelli supervisionati adoperati durante le sessioni sperimentali.

4 Apprendimento Probabilistico

Accanto al comparto deterministico basato sul Gradient Boosting, il sistema *Nutrition Planner* integra un modulo orientato al paradigma dell'apprendimento probabilistico. L'introduzione di tecniche statistiche bayesiane risponde alla necessità ingegneristica di quantificare e modellare l'incertezza intrinseca associata alle predizioni e di valutare la robustezza delle stime relative al dispendio energetico in funzione della variabilità dei dati biometrici e prestazionali degli utenti.

4.1 Regularizzazione e Inferenza Bayesiana: Bayesian Ridge Regression

Per l'implementazione della regressione probabilistica è stato selezionato l'algoritmo **Bayesian Ridge Regression**. Questo approccio estende la classica regressione lineare introducendo distribuzioni di probabilità *prior* (a priori) sui coefficienti del modello, formulando il compito predittivo non più come una stima puntuale fissa, ma in termini di densità di probabilità.

4.1.1 Formulazione Matematica e Scelte Progettuali

Il modello assume che la variabile target y (ovvero *Calories_Burned*) sia distribuita gaussianamente attorno a un valore atteso definito dalla combinazione lineare delle feature X , con un livello di rumore descritto dal parametro α :

$$p(y|X, w, \alpha) = N(y | Xw, \alpha^{-1}I)$$

I coefficienti (o pesi) w del regressore sono a loro volta modellati come variabili sferiche gaussiane regolate da un iperparametro di precisione λ :

$$p(w|\lambda) = \mathcal{N}(w|0, \lambda^{-1}I)$$

A differenza dei tradizionali metodi di regularizzazione (come la Ridge Regression classica), in cui i parametri di penalizzazione α e λ devono essere impostati manualmente tramite costose procedure di validazione incrociata, il framework Bayesian Ridge ottimizza tali parametri stimandoli direttamente dai dati di addestramento durante la fase di massimizzazione della verosimiglianza marginale (*Marginal Likelihood*). Ciò conferisce al modello un'elevata tolleranza al rumore e un'eccellente capacità di prevenire l'overfitting.

4.1.2 Estrazione della Varianza e Intervallo di Confidenza

La caratteristica architetturale primaria del regressore bayesiano risiede nella facoltà di restituire, in fase di inferenza, sia il valore medio predetto della distribuzione (la stima puntuale delle calorie bruciate), sia la relativa deviazione standard o varianza predittiva.

Il software mappa questa informazione statistica per calcolare e presentare all'utente un **intervallo di confidenza**, quantificando matematicamente il grado di precisione del sistema in base alla vicinanza del profilo utente rispetto alla distribuzione dei dati storici appresi. Un valore di varianza ridotto indica una predizione altamente stabile e accurata, mentre una varianza elevata segnala una combinazione insolita o marginale di parametri biometrici, fornendo una preziosa metrica diagnostica a runtime.

4.2 Analisi dei Modelli Scartati: Il Caso SGDClassifier

In fase di progettazione e screening tecnologico è stata valutata la possibilità di integrare un classificatore probabilistico basato sulla discesa stocastica del gradiente, specificamente la classe SGDClassifier di scikit-learn configurata con una funzione di perdita logistica (`loss='log_loss'`), al fine di derivare stime di probabilità su classi discrete di consumo calorico (es. intensità d'allenamento *Bassa, Media, Alta*).

Tuttavia, all'esito dei test empirici di validazione, tale approccio è stato formalmente **scartato** per le seguenti motivazioni metodologiche:

1. **Incompatibilità con la Natura del Target:** Il consumo energetico (`Calories_Burned`) costituisce una grandezza intrinsecamente continua. La sua discretizzazione forzata in classi arbitrarie avrebbe comportato una perdita massiccia di informazione quantitativa, penalizzando la precisione richiesta dal successivo modulo di pianificazione macro-nutrizionale.
2. **Instabilità dell'Ottimizzazione su Piccola Scala:** L'algoritmo SGD (*Stochastic Gradient Descent*) ottimizza i parametri analizzando un singolo record (o mini-batch) alla volta. Sebbene risulti estremamente efficiente su dataset massivi (*Big Data*), su volumi di dati di medie dimensioni manifesta un'elevata instabilità nelle traiettorie di convergenza e una spiccata sensibilità alla calibrazione del tasso di apprendimento (*learning rate*).
3. **Inutilità dell'Approssimazione Lineare:** Nel contesto del sistema *Nutrition Planner*, i modelli *ensemble* non lineari (Gradient Boosting) hanno ampiamente dimostrato prestazioni superiori. L'adozione di un classificatore lineare approssimato tramite SGD non avrebbe apportato alcun beneficio computazionale o predittivo, inducendo una complessità architetturale ingiustificata.

Di conseguenza, per preservare la rigorosa coerenza matematica del framework e massimizzare la precisione delle stime, la pipeline probabilistica del progetto è stata affidata in via esclusiva al modello *Bayesian Ridge*.

5 Knowledge Base Prolog

Accanto al comparto computazionale affidato al Machine Learning, il sistema *Nutrition Planner* integra un modulo logico-deduttivo basato sul paradigma della programmazione dichiarativa, implementato tramite il linguaggio **Prolog** (SWI-Prolog). La Base di Conoscenza (*Knowledge Base - KB*) formalizza i vincoli di dominio del mondo antropometrico e sportivo, assolvendo a due funzioni primarie: validare ed estendere il profilo dell'utente e governare il ragionamento inferenziale per la definizione dei parametri ottimali di allenamento e pasto.

5.1 Struttura della Base di Conoscenza

La Knowledge Base è organizzata in modo modulare per separare i fatti statici dalle regole inferenziali. Essa modella le entità biologiche e prestazionali mediante predicati formali, definendo soglie, coefficienti metabolici e associazioni categoriali.

5.1.1 Definizione dei Fatti

I fatti descrivono le costanti di dominio e le proprietà invarianti degli elementi. Nella KB del sistema, i fatti mappano i livelli di intensità degli allenamenti, i range di età e i moltiplicatori associati al calcolo del tasso metabolico. Esempi chiave di fatti codificati includono:

- Mappatura dei coefficienti per il calcolo del metabolismo basale basati sulla massa magra (secondo formulazioni standard di dominio).
- Classificazione delle categorie di allenamento e dei relativi fattori di impatto fisiologico.

5.1.2 Definizione delle Regole Logiche e Inferenza

Le regole sfruttano l'operatore di implicazione logica ($\leftarrow$) per derivare nuova conoscenza a partire dai dati inseriti dall'utente e dai fatti statici. Il motore di inferenza Prolog valuta i vincoli in cascata per determinare i seguenti profili:

1. **Classificazione della Composizione Corporea**
(**classifica_massa_grassa/3**):classifica_massa_grassa/3 Se l'utente inserisce la propria percentuale di massa grassa, la regola valuta il sesso e l'età per categorizzare il profilo in classi discrete (es. *Atleta, Fitness, Accettabile, Obesità*), applicando i tagli di soglia scientifici:

$$\text{Profilo} \leftarrow f(\text{Sesso}, \text{Età}, \text{Percentuale_Grasso})$$

2. **Stima della Massa Magra (stima_massa_magra/4)**:stima_massa_magra/4 Nel caso in cui il dato sulla massa magra non sia direttamente noto all'utente (valore inserito pari a 0), la KB attiva una regola deduttiva che calcola deterministicamente la massa magra a partire dal peso complessivo e dalla percentuale di massa grassa, restituendo il valore esatto al programma Python.
3. **Pianificazione dell'Intensità e Durata del Workout**
(**consiglio_allenamento/3**):consiglio_allenamento/3 Sulla base dell'obiettivo dichiarato dall'utente (es. *Dimagrimento, Ipertrofia, Mantenimento*) e del suo livello di condizionamento fisico, la regola inferisce l'intervallo di battiti cardiaci target (Avg_BPM) e la durata ideale della sessione, fungendo da vincolo logico prima del passaggio al modello predittivo.

5.2 Interazione tra Python e la Base di Conoscenza

La comunicazione bidirezionale tra l'ambiente procedurale Python e il motore logico Prolog avviene attraverso il bridge architetturale fornito dalla libreria `pyswippyswip`.

La pipeline operativa si articola nelle seguenti fasi:

1. **Validazione e Iniezione dei Dati:** Il modulo Python acquisisce e convalida le metriche biometriche dell'utente (Sesso, Età, Peso, Altezza).
2. **Formulazione della Query Logica:** Python istanzia una stringa di interrogazione formale indirizzata alla KB Prolog (es. `pyswip.Query("classifica_massa_grassa(Sesso, Eta, Grasso, Categoria)")`).
3. **Risoluzione dell'Unificazione:** Il motore Prolog esegue il *backtracking* alla ricerca di tutte le sostituzioni di variabili che rendono l'asserzione vera.
4. **Raccolta dei Risultati:** Python intercetta i dizionari di risposta restituiti da Prolog, estrae i valori unificati e li utilizza per guidare l'esecuzione dei modelli di Machine Learning o per personalizzare i vincoli calorici del piano alimentare.

5.3 Esempio Pratico di Interrogazione (Query)

Per illustrare il funzionamento del motore inferenziale, si riporta un esempio di tracciamento logico relativo alla classificazione antropometrica di un utente di genere maschile (0), di anni 25, con una percentuale di massa grassa pari al 12%.

La query formulata all'interno del sistema si presenta nel seguente formato Prolog standard:

```
Classifica_massa_grassa(0, 25, 12, Categoria).
```

Il motore di inferenza analizza la base dei fatti e le regole relative al genere maschile nella fascia d'età giovanile, verificando che il valore 12 ricade all'interno dell'intervallo ottimale stabilito per la classe atletica. Il processo di unificazione si conclude con successo, restituendo il seguente output:

```
Categoria = 'Atleta' ;  
false.
```

Il software Python interzera la stringa 'Atleta', visualizzandola nel report finale dell'utente e calibrando di conseguenza i successivi algoritmi di ripartizione dei macronutrienti per il piano giornaliero o settimanale.

6 Organizzazione del Codice e Flusso Applicativo

Il cuore operativo dell'applicazione *Nutrition Planner* è interamente codificato nel modulo principale `predict_models.py` **predict_models.py**. Questo script governa l'architettura logica del sistema, orchestrando l'interazione sequenziale tra l'acquisizione dei dati utente, l'esecuzione dei controlli formali, l'interrogazione del motore inferenziale Prolog e l'attivazione dei modelli predittivi di Machine Learning per la successiva sintesi del piano nutrizionale.

6.1 Architettura dei Moduli e Caricamento degli Artefatti

All'avvio dell'applicazione, il sistema provvede all'inizializzazione dell'ambiente e al caricamento in memoria degli artefatti precedentemente serializzati nella fase di addestramento. Questa operazione viene eseguita sfruttando la libreria *joblib* per garantire un'inferenza a bassissima latenza:

1. **Ripristino dello Scaler (*StandardScaler*):** *StandardScaler* Viene caricata l'istanza ottimizzata delle metriche di normalizzazione, indispensabile per convertire i nuovi input dell'utente sulla stessa scala statistica del dataset originario.
2. **Caricamento dei Modelli Regressori:** Vengono allocati in memoria i file binari corrispondenti al modello ensemble primario (*Gradient Boosting*) e al modello probabilistico (*Bayesian Ridge*).
3. **Inizializzazione della KB:** Viene istanziato il modulo di collegamento con il file *Prolog* contenente le regole deduttive descritte nel capitolo precedente.

6.2 Validazione degli Input e Controllo dei Vincoli

Al fine di garantire l'integrità matematica delle predizioni ed evitare l'insorgenza di eccezioni a runtime, il modulo implementa una rigida routine di validazione interattiva degli input antropometrici e fisiologici. Ciascun dato inserito dall'utente viene sottoposto a controlli vincolati entro range realistici di dominio medico-sportivo:

- **Sesso:** Accetta esclusivamente i valori discreti 0 (Maschio) o 1 (Femmina).
- **Età e Peso:** Verificati all'interno di intervalli biologici plausibili.
- **Altezza:** Vincolata a valori strettamente positivi espressi in metri.
- **Frequenza Cardiaca (*Avg_BPM*):** *Avg_BPM* Sottoposta a controllo di coerenza per riflettere i limiti fisiologici di un battito cardiaco medio sotto sforzo.

Qualora l'algoritmo rilevi un input non conforme o logicamente incoerente (es. percentuali di massa grassa e magra matematicamente incompatibili), il flusso d'esecuzione inibisce il passaggio ai modelli, visualizza un messaggio d'errore localizzato e richiede iterativamente l'inserimento del dato corretto.

6.3 Algoritmi di Generazione del Piano Alimentare

Una volta validate le metriche e calcolato il dispendio calorico totale — combinando il Metabolismo Basale (BMR) derivato tramite equazioni scientifiche (come la formula di *Katch-McArdle* basata sulla massa magra fornita da *Prolog*) e il surplus energetico del workout predetto dal *Gradient Boosting* — il sistema attiva il motore di pianificazione nutrizionale sul dataset *alimenti.csv*.

6.3.1 Ripartizione dei Macronutrienti e Calcolo delle Grammature

Il software traduce il target calorico giornaliero in precisi obiettivi di macronutrienti (Proteine, Carboidrati, Grassi), applicando percentuali di ripartizione standardizzate in base all'obiettivo dell'utente. Il modulo calcola analiticamente le grammature necessarie partendo dal presupposto che:

- 1 grammo di Proteine o Carboidrati apporta circa 4 kcal .

- 1 grammo di Grassi apporta circa 9\text{ kcal}.

L'algoritmo effettua una ricerca combinatoria sul dataset degli alimenti per selezionare le referenze idonee e calcolarne le grammature esatte su base 100g, minimizzando lo scarto rispetto al target stabilito.

6.3.2 Struttura del Pasto e Generazione delle Varianti

Il piano alimentare viene strutturato suddividendo la giornata in quattro portate principali stoccate in strutture dati dizionario: **Colazione**, **Pranzo**, **Spuntino** e **Cena**. Il sistema supporta due modalità operative di generazione:

- **Alternative Giornaliere (Varianti):** Quando l'utente richiede l'elaborazione di un singolo giorno, l'algoritmo introduce un indice di variante stocastico. Questo meccanismo permette di variare ciclicamente gli alimenti selezionati, garantendo che a parità di apporto calorico e macronutrizionale l'utente riceva combinazioni culinarie differenti e non ripetitive.
- **Piano Settimanale:** Estende la logica di generazione su un orizzonte temporale di sette giorni (da Lunedì a Domenica). Il modulo cicla sequenzialmente sulle portate applicando variazioni sistematiche nei filtri di selezione del tipo di alimento, strutturando un palinsesto dietetico variegato e bilanciato che previene la ridondanza dei pasti attingendo dinamicamente dall'intero spettro del database alimentare.

The screenshot shows a code editor with Python code for a Nutrition Planner. The code includes imports for `os`, `joblib`, and `pandas`, and defines a `BASE_DIR` variable. The terminal output displays the results of the inference and predictions, including recommended workout, estimated intensity, duration, and various regression models for calorie prediction. It also shows the recommended daily meal plan with macronutrient breakdown and a 5-day meal plan for Monday.

```

1 You, yesterday | 1 author (You)
2 import os
3
4 import joblib
5 import pandas as pd
6
7 from nutrition_planner import (
8     ask_float,
9     ask_int,
10    build_nutrition_plan,
11    print_meal_plan,
12 )
13
14 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
15
16

```

```

christiandomenicotermin@MacBook-Air-di-Christian Nutrition Planner % /usr/bin/python3 "/Users/christiandomenicotermin/Desktop/Nutrition Planner/predict_models.py"

=== Risultati dell'Inferenza Prolog ===
Workout consigliato: Cardio
Intensita' stimata: high
Durata ottimale: 3.0 ore

=== Risultati delle Predizioni ===
Regressione Lineare: 1967.53 kcal
Random Forest: 1063.22 kcal
Gradient Boosting: 1321.66 kcal
Bayesian Ridge: 1967.34 kcal
Intervallo di Confidenza Bayesian Ridge: [1887.52, 2047.15] kcal

=== Piano Alimentare Consigliato ===
Obiettivo stimato: dimagrimento
Metabolismo basale: 1481.97 kcal
Fabbisogno calorico totale: 2297.05 kcal
Calorie giornaliere consigliate: 1952.49 kcal

=== Macronutrienti Giornalieri ===
Proteine: 113.26 g
Carboidrati: 234.47 g
Grassi: 62.40 g

=== Piano Alimentare per 5 Giorni ===
--- Lunedì | 1889.80 kcal | P 144.88 g | C 236.63 g | G 46.73 g ---

```

- *Figura 6.1: Output finale stampato sulla CLI di sistema dal modulo `predict_models.py`, mostrante lo specchietto riassuntivo del piano alimentare calcolato.*

7 Installazione ed Esecuzione del Sistema

Per consentire la portabilità del framework *Nutrition Planner* e assicurarne la corretta replicabilità in ambienti locali operativi, le procedure di installazione delle dipendenze software e di avvio dell'applicazione sono state standardizzate mediante l'uso di interfacce a riga di comando (CLI) e file di configurazione delle metriche di ambiente.

7.1 Prerequisiti di Sistema

Prima di procedere con l'installazione dei moduli Python, è necessario verificare la presenza dei seguenti componenti fondamentali nel sistema ospite:

- **Python (v3.10 o superiore):** Interprete principale del codice procedurale e dei framework di Machine Learning.
- **SWI-Prolog:** Il motore di esecuzione logica e computazione inferenziale. L'eseguibile di SWI-Prolog deve essere correttamente installato nel sistema e configurato all'interno delle variabili d'ambiente globali (PATH), consentendo alla libreria pyswip di agganciare dinamicamente i pacchetti dinamici della libreria condivisa del kernel Prolog (libswipl).

7.2 Installazione delle Dipendenze

All'interno della directory principale del progetto è presente il file di configurazione strutturato `requirements.txt`, all'interno del quale sono dichiarate in modo analitico tutte le librerie software di terze parti necessarie all'esecuzione dei moduli (es. scikit-learn, pandas, numpy, joblib, pyswip).

L'allineamento dell'ambiente virtuale o locale può essere eseguito lanciando il seguente comando all'interno del terminale di sistema:

```
pip install -r requirements.txt
```

Questo comando attiva il gestore di pacchetti di Python, che provvede al download automatico e alla risoluzione delle versioni compatibili di ciascuna dipendenza, azzerando i conflitti di compilazione.

7.3 Flusso di Esecuzione e Avvio dell'Applicazione

Una volta validati i prerequisiti e compilato l'ambiente software, l'utente può inizializzare l'interfaccia interattiva del sistema invocando lo script principale dal terminale posizionato nella cartella radice:

```
python predict_models.py
```

All'atto del lancio, l'applicazione esegue in sequenza le seguenti macro-operazioni architetturali:

1. **Verifica della KB:** Controlla l'accessibilità del file Prolog e compila le regole di sottomissione dei vincoli.
2. **Bootstrap dei Modelli:** Alloca in memoria lo scaler normalizzato e i regressori serializzati (Gradient Boosting e Bayesian Ridge).

3. **Ingestione del Profilo:** Avvia il ciclo interattivo di domande sul terminale per l'acquisizione validata dei dati antropometrici dell'utente.
4. **Output dei Risultati:** Presenta a schermo l'inferenza probabilistica della varianza calorica del workout e stampa la griglia strutturata del piano alimentare (giornaliero con varianti o settimanale completo), formattandone le grammature per singolo pasto.

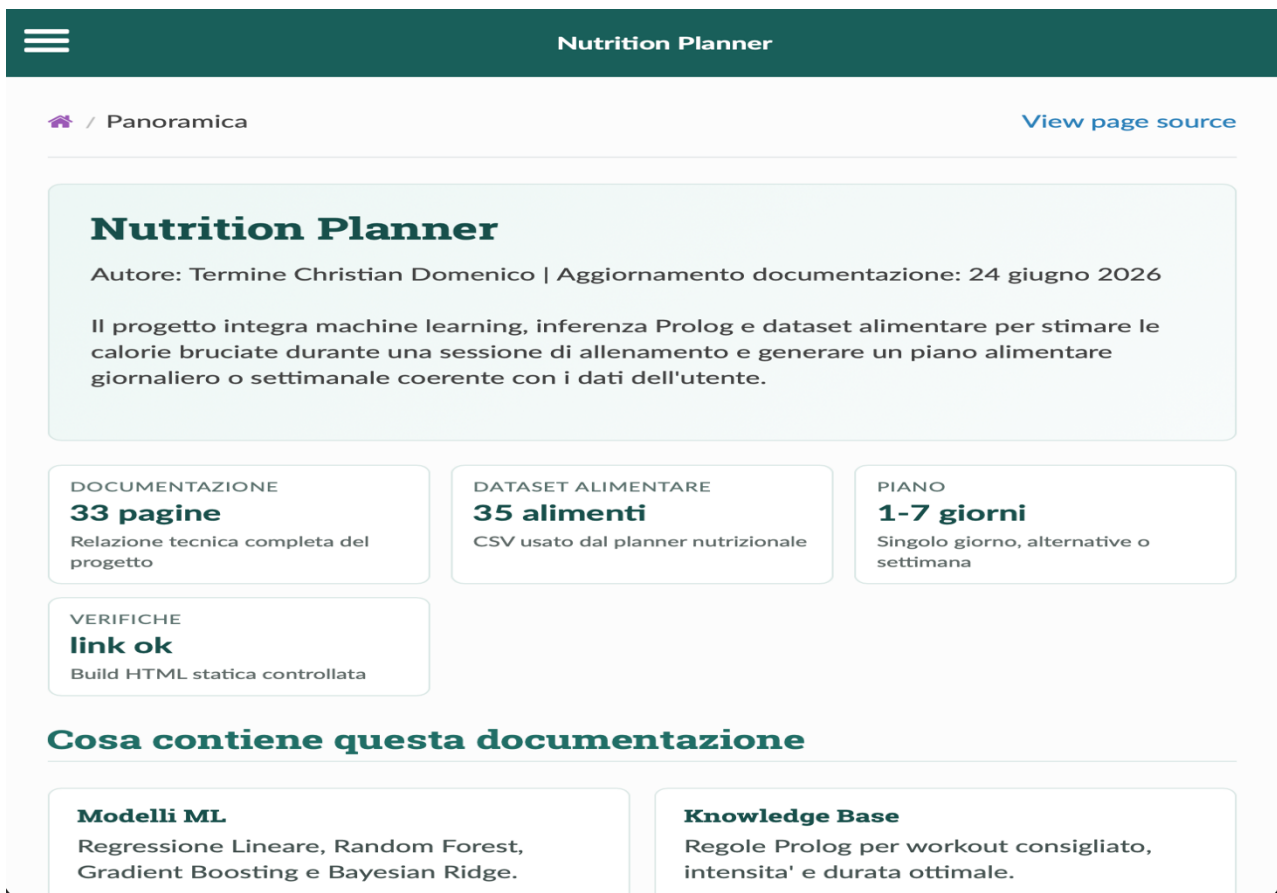
8 Documentazione e Repository GitHub

Al fine di garantire la manutenibilità, l'estendibilità e la massima trasparenza del codice sviluppato per il sistema *Nutrition Planner*, è stata prodotta una documentazione tecnica esaustiva sia per il comparto procedurale e di Machine Learning in Python, sia per la componente logico-deduttiva strutturata in Prolog.

8.1 Documentazione del Comparto Python (Sphinx)

La documentazione del codice sorgente Python è stata ingegnerizzata sfruttando il framework standard di documentazione intelligente **Sphinx**. Mediante l'analisi automatica delle *docstring* e dei commenti strutturati inseriti nei vari moduli (incluso lo script primario `predict_models.py`), è stata compilata una build ipertestuale completa:

- **Collocazione degli Artefatti:** I file sorgente e i file di configurazione di Sphinx risiedono all'interno della directory `docs/`.
- **Accesso alla Build:** All'interno della sottocartella `docs/_build/html/`, il punto di accesso ipertestuale principale è rappresentato dal file `index.html`. Aprendo tale file tramite un qualsiasi browser web, è possibile navigare un'interfaccia interattiva e navigabile che descrive analiticamente l'indice dettagliato delle funzioni, la firma dei moduli, la tipizzazione delle strutture dati di input/output, i dettagli del dataset alimentare e le funzioni di controllo dei vincoli per il planner nutrizionale.



- *Figura 8.1: Interfaccia ipertestuale della documentazione dei moduli Python generata tramite il framework Sphinx.*

8.2 Documentazione della Knowledge Base Prolog (PIDoc)

La documentazione della componente dichiarativa è stata interamente affidata al sistema nativo **PIDoc** di SWI-Prolog. Questo strumento consente di formalizzare la semantica dei predicati logici direttamente all'interno della Knowledge Base:

- **Collocazione degli Artefatti:** La documentazione ipertestuale generata è ospitata all'interno della cartella dedicata `kb/doc/kb/doc/`.
- **Consultazione delle Regole:** Il file principale `index.html` funge da indice e raccoglie in tabelle descrittive strutturate la documentazione di tutti i fatti statici, le condizioni di unificazione e i predicati logico-deduttivi esportati dal modulo delle regole (come `classifica_massa_grassa/4` o `stima_massa_magra/4`), specificandone i modi di istanziazione e i vincoli di dominio.

 Nutrition Planner

 / Knowledge Base Prolog [View page source](#)

Knowledge Base Prolog

La cartella `kb` contiene `kb.pl`, usato da `predict_models.py` tramite `pyswip`. La base di conoscenza non genera il piano alimentare: fornisce indicazioni su `workout`, `intensita'` e `durata` ottimale.

Documentazione Prolog consultabile su GitHub: <https://github.com/Christian032003/Nutrition-Planner/tree/main/kb/doc>

Predicato	Descrizione
<code>recommended_workout/3</code>	Consiglia il workout partendo da peso e altezza tramite BMI e <code>fitness_level</code> .
<code>recommended_intensity/4</code>	Stima l' <code>intensita'</code> in base alle calorie bruciate previste.
<code>optimal_duration/4</code>	Regola la durata proposta in base all' <code>intensita'</code> stimata.
<code>calories_per_kg/2</code>	Fatti Prolog con consumo calorico per kg e ora per Cardio, HIIT, Strength e Yoga.

Query eseguite dal programma

```
recommended_workout(Weight, Height, Workout)
recommended_intensity(Weight, Height, Intensity, Duration)
optimal_duration(Weight, Height, Duration, OptimalDuration)
```

 Previous

Next 

- *Figura 8.2: Consultazione dei predicati logici della Knowledge Base mediante l'indice generato da PIDoc.*

8.3 Repository GitHub del Progetto

Il codice sorgente dell'intero ecosistema *Nutrition Planner*, unitamente ai dataset in formato CSV, agli artefatti binari serializzati dei modelli e alle suite di documentazione ipertestuale, è tracciato e pubblicato in modalità open-source su piattaforma di *version control* GitHub.

Il repository ufficiale è raggiungibile al seguente indirizzo:

- **Link GitHub:** <https://github.com/Christian032003/Nutrition-Planner>

Il repository è strutturato in modo da riflettere fedelmente la topologia delle cartelle descritta nella relazione, includendo un file README.md esplicativo per agevolare il deployment rapido del sistema da parte della community scientifica o di terzi sviluppatori.

9 Riferimenti Bibliografici

- **Kaggle Platform:** Portale di riferimento per il recupero del dataset open-source sul monitoraggio dei workout e dei parametri biologici degli utenti (*gym_exercise_dataset.csv*). <https://www.kaggle.com/>
- **David L. Poole, Alan K. Mackworth** – *Artificial Intelligence: Foundations of Computational Agents* – Cambridge University Press (2010). Testo di riferimento per i paradigmi di rappresentazione della conoscenza e i sistemi basati su regole logiche.
- **Scikit-learn Documentation:** Documentazione ufficiale per la calibrazione iperparametrica tramite GridSearchCV, i modelli ensemble (*Gradient Boosting*, *Random Forest*) e la regressione probabilistica bayesiana. <https://scikit-learn.org/>
- **SWI-Prolog Reference Manual:** Documentazione di riferimento per lo sviluppo di basi di conoscenza dichiarative e per l'integrazione del motore d'inferenza con linguaggi esterni mediante interfacce applicative. <https://www.swi-prolog.org/>